

SCHEDULING ALGORITHMS :

1. fcfs :

```
mey@localhost ~  
[mey@localhost ~]$ ls  
bank  Deshtep  Desktop  davan  Music  Pictures  sh  Videos  
a     banker.c  Documents devan.c os_lab  Public    sjf  while.sh  
ABC   bankers  Downloads dev     os_lab1 BEC       sjf.c  while.sh  
abc.sh bm       h       dev.c   os_lab1.c sample.c Templates XYZ  
ab.sh close  dev     devan   os_lab.c sample.txt test.sh  
[mey@localhost ~]$ vi bestfit.c  
[mey@localhost ~]$ gcc bestfit.c -o bestfit  
[mey@localhost ~]$ ./bestfit  
Enter number of processes: 3  
Enter Arrival Time and Burst Time for P1: 0 5  
Enter Arrival Time and Burst Time for P2: 1 3  
Enter Arrival Time and Burst Time for P3: 2 8  
  
Best Fit Average Turnaround Time: 8.67  
Best Fit Average Waiting Time: 3.33  
[mey@localhost ~]$
```

```
Activities Terminal Mar 18 19:12  
mey@localhost:~ — /usr/bin/vi bes  
[mey@localhost ~]$ vi bestfit.c  
int main() {  
    int n, 4, i;  
  
    printf("Enter number of processes: ");  
    scanf("%d", &n);  
  
    int at[n], bt[n], ct[n], tat[n], wt[n];  
  
    // Input  
    for(i = 0; i < n; i++) {  
        printf("Enter Arrival Time and Burst Time for P%d: ", i+1);  
        scanf("%d %d", &at[i], &bt[i]);  
    }  
  
    float tatat_tat = 0, total_wt = 0;  
  
    // First process  
    ct[0] = at[0] = at[0];  
  
    // Remaining processes  
    for(i = 1; i < n; i++) {  
        if(at[i] < at[i-1])  
            ct[i] = at[i] - bt[i-1]; // CPU idle  
        else  
            ct[i] = ct[i-1] - bt[i-1];  
    }  
  
    // Calculate TAT and WT  
    for(i = 0; i < n; i++) {  
        tat[i] = ct[i] + bt[i];  
        wt[i] = tat[i] - bt[i];  
  
        total_tat += tat[i];  
        total_wt += wt[i];  
    }  
  
    printf("\nAverage Turnaround Time: %.2f", total_tat / n);  
    printf("\nAverage Waiting Time: %.2f", total_wt / n);  
  
    return 0;  
}
```

2. sjf :

```
mey@localhost:~$ ls
..      bank      Desktop  matthu  Nueic   Pictures sh      whilew.sh
a       devan.c  Documents devan   os_lab  Public  Templates XVZ
sjf     bankers  Downloads maitre  os_lab1 RFC     trat.sh
abc.sh  hm       h        monb    os_lobl.c sample.c RTDees
ab.sh   close    sjf      mtch11  os_lab.c sample.tss while.sh

[mey@localhost ~]$ vi sjf
[mey@localhost ~]$ vi sjf.c
[mey@localhost ~]$ vi sjf.c
[mey@localhost ~]$ vi sjf.c
[mey@localhost ~]$ gcc sjf.c -o sjf
[mey@localhost ~]$ ./sjf
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 8 7
Enter Arrival Time and Burst Time for P2: 2 4
Enter Arrival Time and Burst Time for P3: 4 1

Average Turnaround Time: 7.00
Average Waiting Time: 3.80
[mey@localhost ~]$
```

Code :

```
Activities Terminal Mar 18 19:10 mey@localhost: ~/usr/bin/vim sjf.c

#include <stdio.h>
int main()
{
    int a;
    printf("Enter number of processes: ");
    scanf("%d", &a);

    int at[a], bt[a], tat[a], wt[a];
    int done[a];

    // Input
    for(int i = 0; i < a; i++)
    {
        printf("Enter Arrival Time and Burst Time for P%d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        done[i] = 0;
    }

    int completed = 0;
    int completedTime = 0;
    int augtet = 1, arget = 0;

    // SJF Scheduling (Non-Preemptive)
    while(completed < a)
    {
        int tds = -1;
        int wnbBT = 0;

        for(int i = 0; i < a; i++)
        {
            if(pr[i] <= completionTime && done[i] == 0)
            {
                if(dsti < wnbBT)
                {
                    wnbBT = bt[i];
                    tds = i;
                }
            }
        }

        if(tds < -1)
    }
}
```

```
Activities Terminal Mer 18 19:10
mey@localhost: ~ - /usr/bin/vim sjf.c

if(st[i] <= completionTime && done[i] == 0)
{
    if(st[i] < minST)
    {
        minST = st[i];
        idx = i;
    }
}

if(idx != -1)
{
    if(completionTime < st[idx])
        completionTime = st[idx];

    completionTime += bt[idx];

    tat[idx] = completionTime - at[idx];
    wt[idx] = tat[idx] - bt[idx];

    done[idx] = 1;
    completed++;
}
else
{
    completionTime++;
}

for(int i = 0; i < n; i++)
{
    avgst += tat[i];
    wgt += wt[i];
}

printf("Average Turnaround Time: %.2f\n", avgst / n);
printf("Average Waiting Time: %.2f\n", wgt / n);

return 0;
}
[mey@localhost ~]$
[mey@localhost ~]$
```

3. Round robin :

```
Activities Terminal Mar 18 29:07
mey@localhost:~$ ls
. bank Desktop mait Music Pictures sh Videos
s banker.c Documents maitlab es_lab Public sjf write.sh
MEC banbars Downloads maitlab es_lab REC sjf.s write.sh
abc.sh bin h maitlab es_lab.c sample.c TopAntes YVZ
eb.sh close maitlab es_lab.c sample.txt test.sh
[mey@localhost ~]$ vi vi roundrobin.c
[mey@localhost ~]$ ls gcc roundrobin.c -o roundrobin
[mey@localhost ~]$ ls ./round robin
bash: ./round: No such file or directory
[mey@localhost ~]$ +s ./roundrobin
Enter number of processes:
3
Enter Time Quantum:
2
Enter AT and BT for P1:
0 5
Enter AT and BT for P2:
1 3
Enter AT and BT for P3:
2 1

Average Turnaround Time: 6.23
Average Waiting Time: 3.33
[mey@localhost ~]$
```

Code :

```
Activities Terminal Mar 18 10:08
mey@localhost: ~/usr/bin/vim reundrebin.c

data: void *cddia, int
void main()
{
    int t, to, completed = 0, completionTime = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    int at[n], bt[n], rem[n], tat[n], wt[n];
    float avgat = 0, avgwt = 0;

    // Round Robin
    for (int i = 0; i < n; i++)
    {
        printf("Enter AT BT for %d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
        rem[i] = bt[i]; // Remaining time - burst time
    }

    // Round Robin
    while (completed < n)
    {
        int idle = 1;

        for (int i = 0; i < n; i++)
        {
            if (at[i] == completionTime && rem[i] > 0)
            {
                idle = 0;

                if (rem[i] > tq)
                {
                    completionTime += tq;
                    rem[i] -= tq;
                }
                else
                {
                    completionTime += rem[i];
                }
            }
        }

        if (idle)
            completionTime++;
    }

    // Calculate averages
    for (int i = 0; i < n; i++)
    {
        avgat += tat[i];
        avgwt += wt[i];
    }

    printf("Average Turnaround Time: %.2f\n", avgat / n);
    printf("Average Waiting Time: %.2f\n", avgwt / n);

    return 0;
}
```

```
Activities Terminal Mar 18 10:08
mey@localhost: ~/usr/bin/vim reundrebin.c

int idle = 1;
for (int i = 0; i < n; i++)
{
    if (at[i] == completionTime && done[i] == 0 && rem[i] > 0)
    {
        idle = 0;
        if (rem[i] > tq)
        {
            completionTime += tq;
            rem[i] -= tq;
        }
        else
        {
            completionTime += rem[i];
            tat[i] = completionTime - at[i];
            wt[i] = tat[i] - bt[i];
            rem[i] = 0;
            done[i] = 1;
            completed++;
        }
    }
}

if (idle)
    completionTime++;

// Calculate averages
for (int i = 0; i < n; i++)
{
    avgat += tat[i];
    avgwt += wt[i];
}

printf("Average Turnaround Time: %.2f\n", avgat / n);
printf("Average Waiting Time: %.2f\n", avgwt / n);

return 0;
[mey@localhost ~]$
```

4. priority scheduling

```

mer@localhost:~$ ls
banker.c  Downloads  mch  os_lab.c  sample.c  test.sh
a         bankers  h    mt011     Privates  sample.txt  Videos
shc       ha       mt    Nexts     Public    sh          akile.sh
abc.sh    clesse  dev   es_lab    KGI       a/f        wkfle.sh
ab-ch     Decktop  deven es_lab1    roundrablo s/f.c      it2
bank      Documents devan  es_lab1.c roundrablo templates

mer@localhost:~$ vi priority
mer@localhost:~$ vi priority.c
mer@localhost:~$ gcc priority.c -o priority
mer@localhost:~$ ./priority
Enter number of processes:
3
Enter AT, BT and Priority for P1:
0 5 2
Enter AT, BT and Priority for P2:
1 3 1
Enter AT, BT and Priority for P3:
2 8 3

Average Turnaround Time: 6.67
Average Waiting Time: 3.33
mer@localhost:~$

```

```

Activities Terminal Mar 18 19:19
mey@localhost:~$ vim priority.c

#include <stdio.h>

int main()
{
    int n, completed = 0, time = 0;

    printf("Enter number of processes: \n");
    scanf("%d", &n);

    int arr[100], br[100], priority[100], done[100];
    int start[100], end[100], ed[100];
    float avgwt = 0, avgct = 0;

    // Input
    for (int i = 0; i < n; i++)
    {
        printf("Enter ST, BT and Prio for Proc %d\n", i + 1);
        scanf("%d %d %d", &start[i], &end[i], &priority[i]);
        done[i] = 0;
    }

    // Scheduling Logic (non-preemptive priority: lower number = higher priority)
    while (completed != n)
    {
        int tdx = -1;
        int highPriority = 9999;

        for (int i = 0; i < n; i++)
        {
            if (start[i] <= time && done[i] == 0)
            {
                if (priority[i] < highPriority)
                {
                    highPriority = priority[i];
                    tdx = i;
                }
            }
        }

        if (tdx != -1)
        {
            time = end[tdx];
            done[tdx] = 1;
            completed++;
            avgwt += (time - start[tdx]) / n;
            avgct += 1 / n;
        }
    }

    printf("Average waiting time: %f\n", avgwt);
    printf("Average context switching: %f\n", avgct);
}

```

```
Activities Terminal Nov 15 15:13
mey@localhost:~/usr/bin/vim priority.c

if (ids[i] == -1)
{
    time += 1; // 100;
    cr[100] = 100;
    test[100] = cr[100] - ar[100];
    ar[100] = test[100] - 80[100];

    for[100] = 1;
    compare++;
}
else
{
    time++; // 0% 2.5e
}
}

// Average calculation
for (int i = 1; i < n; i++)
{
    avgat = tat[i];
    avgat = at[i];
}

printf("Average Turnaround Time: %.2f\n", avgat / n);
printf("Average Waiting Time: %.2f\n", avgat / c);

return 1;
}
```